

SOMA: Sovereign Operative Memory Architecture

A Cognitive Operating System for Long-Horizon Autonomous Agents

Mario Raúl Carbonell Martínez (mcarbonell)

Independent Researcher

github.com/mcarbonell/soma

March 2026

Abstract

Large language models are effective local reasoners but poor long-horizon operators. In demanding environments such as software engineering, compilation, or game-playing, the dominant failure mode is not insufficient intelligence but the absence of bounded, persistent, and inspectable cognitive state. Most current agent stacks address this with increasingly complex orchestrators, growing chat transcripts, or retrieval pipelines that remain external to the model’s own decision loop.

This paper introduces SOMA (**Sovereign Operative Memory Architecture**), a four-layer cognitive operating system for autonomous agents. SOMA separates bounded working attention (**L1**), immutable episodic traces (**L2**), persistent sovereign knowledge (**L3**), and environment grounding (**L4**). The architecture makes context saturation explicit through a prompt-occupancy signal called *Somatic Pressure* (P_m), and gives the agent direct metacognitive affordances to checkpoint, distill, and reorganize its own working state without external intervention.

We present the reference JavaScript runtime, a suite of empirical benchmarks including the *Leviathan* series of extreme long-horizon tasks, and three research directions that extend the architecture toward contextual memory injection, a synthetic hippocampus for sub-millisecond intuition, and a sovereign inference kernel capable of mutating working memory during generation. Empirical results show that a SOMA agent running on a free-tier language model completes a from-scratch C compiler in 132 turns and a fully verified chess engine (Perft depth 1–5, 99.9997% accuracy) in 99 turns, both with P_m consistently below 20%.

1 Introduction

Recent progress in language agents has demonstrated that large language models can plan, call tools, browse, write code, and interact with external environments [1, 5, 2]. Yet production agent systems still exhibit a familiar pathology: they accumulate raw transcripts, debugging detours, and stale observations inside a single context window until the model degrades, drifts, or stalls. In software engineering settings this failure mode is especially acute because the environment is dense, stateful, and constantly changing.

The standard response has been to strengthen the orchestrator. The external runtime decides what to summarize, what to retrieve, when to prune context, and how to route tools. This can work, but it creates what we call the *orchestrator’s ceiling*: the agent remains bounded by heuristics hard-coded outside the model’s own cognitive loop. The more autonomy we want, the more brittle this arrangement becomes.

SOMA starts from a different premise. The long-horizon problem is best treated as an *operating-system problem for cognition* rather than as a prompt-engineering problem. The architecture exposes to the model a small set of explicit substrates: a bounded attention space, an immutable action ledger, a persistent sovereign knowledge store, and a structured interface

to the external world. The orchestrator remains intentionally thin—its role is to expose the machine and preserve state, not to out-think the model. This design philosophy is directly inspired by Sutton’s *The Bitter Lesson* [10]: the history of AI shows that methods which leverage computation and learning at scale consistently outperform methods that encode human knowledge into the system. Applied to agent orchestration, the lesson is that hard-coded heuristics in the orchestrator are a ceiling; the model’s own reasoning, given the right substrates and signals, scales with compute.

A key design insight is that *more context is not equivalent to better cognition*. Research on transformer attention has shown that filling a large context window with low-value traces causes “context rot” and the “lost in the middle” phenomenon: the model becomes simultaneously more expensive and less accurate [9]. SOMA was designed around this premise from the start: L1 is kept surgically small, and the agent itself decides what to promote to persistent storage and what to discard.

This paper makes five contributions:

1. A four-layer memory and environment architecture for long-horizon agents.
2. *Somatic Pressure* (P_m), an explicit prompt-occupancy signal for self-regulated context management.
3. A reference runtime in which the agent checkpoints and distills its own state instead of relying on hidden orchestration.
4. The *Leviathan* benchmark series: extreme long-horizon tasks (C compiler, bytecode VM, chess engine) with deterministic evaluators.
5. Three forward-looking research directions grounded in the operational experience of running the system.

2 Architecture

SOMA organizes agent cognition into four layers. Table 1 summarizes their roles and current realizations.

Layer	Role	Current realization
L1	Bounded working attention	Structured prompt assembled each turn from identity, task, notes, relevance-ranked file tree, environment panels, recent milestones, retrieved memories, and P_m dashboard
L2	Immutable episodic record	Session-scoped JSONL action ledger and milestone history in <code>~/.soma/L2/sessions/</code>
L3	Sovereign persistent knowledge	Identity, semantic memory, file annotations, knowledge tree, and persistent agent state in <code>~/.soma/L3/</code>
L4	User environment	Real workspace plus <code>.soma-workspace/</code> metadata; IDE suite with file, terminal, git, and browser panels

Table 1: The four layers of SOMA.

Figure 1 illustrates the data flow between layers and the role of P_m as a feedback signal.

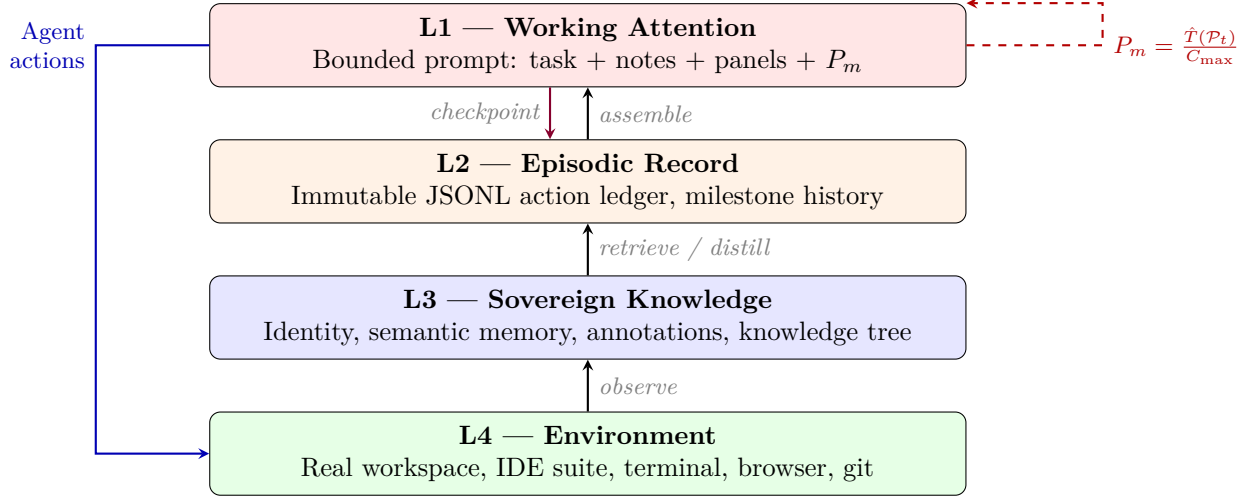


Figure 1: Data flow in the SOMA architecture. Observations flow upward from the environment (L4) through persistent knowledge (L3) and the episodic ledger (L2) into bounded working attention (L1). Agent actions flow downward. The dashed loop represents the P_m self-regulation signal.

2.1 L1: Bounded Working Attention

L1 is not the chat history. It is the current working set, assembled dynamically each turn. The agent sees an intentionally bounded surgical table rather than an unbounded transcript. Crucially, the agent itself controls what enters L1: it can open files, expand folders, retrieve memories, and pin notes. It can also close files, forget memories, and commit milestones to clear the action log—all of which reduce P_m .

2.2 L2: Immutable Episodic Record

L2 stores what the agent did, when, and with what rationale. Unlike ordinary chat logs, L2 is not the default attention substrate. It is an append-only ledger used for continuity, audit, and later distillation. A failed command or long terminal output may be preserved in L2 for audit without remaining resident in L1.

2.3 L3: Sovereign Knowledge

L3 is the agent’s long-lived substrate. It persists across projects and sessions. The agent can write structured knowledge files, annotate paths with semantic tags, and index memories with embeddings for later retrieval. The design favors continuity and cross-project transfer over strict isolation.

2.4 L4: Environment

L4 is the real operating environment. A key distinction of SOMA is that the environment is not a generic “external world” abstraction—it is a first-class layer with adapter-specific semantics. The current reference runtime exposes an IDE suite. The same layer could be instantiated through simulator state, robotic sensors and actuators, or other domain-specific interfaces.

2.5 Core, Adapters, and Applications

We distinguish three levels to avoid conflating the architecture with its first implementation:

- **SOMA-Core:** the domain-general cognitive OS consisting of L1–L4 semantics, P_m , checkpointing, distillation, and persistent identity.
- **Environment adapters:** concrete realizations of L4 (IDE, simulator, robotic sensorimotor, supervisory interface).
- **Applications:** task-specific systems built on the core and one or more adapters.

The current JavaScript artifact is a validated software application of SOMA-Core, not the total extent of the architecture.

2.6 Operational Loop

Each turn the agent executes:

1. Observe L4 through tool-mediated environment panels.
2. Assemble L1 from task state, metacognition, recent milestones, and retrieved L3 knowledge.
3. Execute one or more actions in L4 or internal memory.
4. Append an immutable trace to L2.
5. When stable progress is reached or P_m rises, checkpoint and distill selected traces into L3.

2.7 Hive Mind: Sub-Agent Delegation

SOMA v7.0 introduced *Hive Mind*: the ability for the primary agent to hire sub-agents via `hire_sub_agent(tier, task, budget)`. Sub-agents receive the full toolkit except `hire_sub_agent` itself, preventing recursive delegation. The recursion guard is implemented via an `isSubAgent` flag on the SOMA instance; the orchestrator blocks the tool call at dispatch time rather than relying on prompt-level instructions. This design separates the *policy* (no sub-agent may hire) from the *mechanism* (runtime enforcement), making it robust to prompt injection.

3 Somatic Pressure and Self-Regulation

3.1 Motivation

Long-horizon agents often fail because context saturation is invisible to the model. The orchestrator notices only indirectly, typically after reasoning quality has already degraded. SOMA makes saturation explicit through *Somatic Pressure* (P_m).

3.2 Definition

P_m is defined as prompt occupancy:

$$P_m(t) = \frac{\hat{T}(\mathcal{P}_t)}{C_{\max}} \quad (1)$$

where $\hat{T}(\mathcal{P}_t)$ is the estimated token count of the full prompt assembled at turn t , and $C_{\max}$ is the model context capacity. To provide real-time feedback without the latency of an external API call, the reference implementation uses a character-based heuristic ($\hat{T} \approx \text{chars}/4$), which provides a consistent and model-agnostic relative signal. Actual billing tokens are tracked separately via API responses for cumulative reporting.

This definition ties pressure to the real artifact. SOMA measures the complete prompt the model actually receives—including file tree, knowledge summaries, milestones, environment context, and injected memories—not just fragments such as notes or task text.

3.3 Self-Regulation Thresholds

The agent sees P_m as a visual bar in its L1 dashboard on every turn:

- $P_m < 75\%$: green. Normal operation.
- $75\% \leq P_m < 85\%$: yellow. Agent is advised to consider `commit_milestone`.
- $P_m \geq 85\%$: red. Agent receives an urgent directive to checkpoint immediately.

Crucially, the *agent* decides when to checkpoint—the orchestrator only surfaces the signal. This keeps policy inside the cognitive loop rather than hard-coded outside it.

3.4 Checkpoint and Distillation

When the agent calls `commit_milestone`, the action log is cleared from L1, the milestone is appended to the L2 session record, and stable conclusions are promoted to L3 notes or knowledge files. The objective is not compression but *reorganization*: transient traces remain available in L2 for audit, while high-value conclusions become cheap to reactivate in future turns.

Figure 2 shows the P_m trajectory versus the cumulative token count during the *Leviathan TinyC Compiler* benchmark. While the cumulative tokens grow monotonically to nearly 1.9 million, the agent maintains P_m below the 20% boundary through active context management.

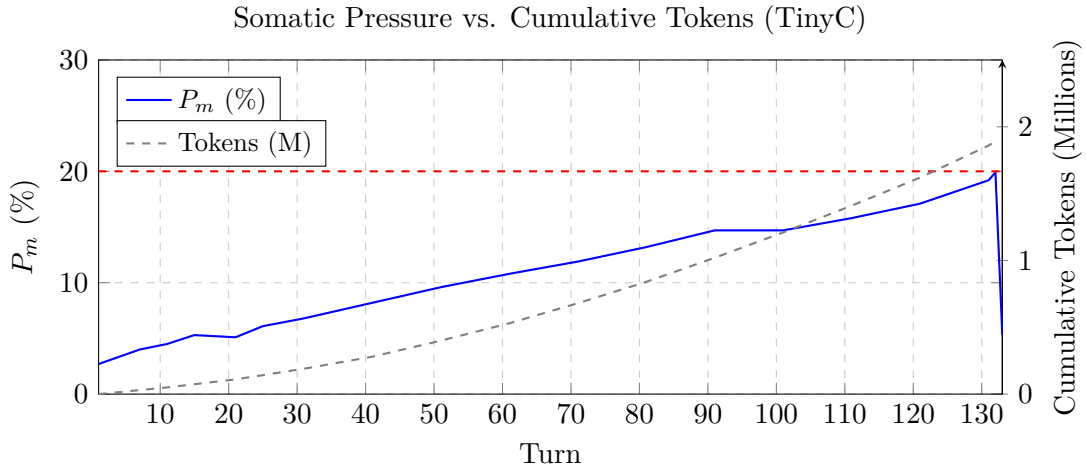


Figure 2: The P_m trajectory (blue) remains stable despite the monotonic growth of cumulative tokens (gray dashed). The sharp drop at turn 133 represents the L1 cleanup after the final `commit_milestone`.

4 Leviathan Benchmarks

4.1 Design Philosophy

Standard agent benchmarks tend to be short-horizon (single tool call, single file, single answer) or evaluated by a judge model rather than a deterministic oracle. Neither design stresses the memory management properties that SOMA is built to address.

The *Leviathan* series was designed around three principles:

1. **Deterministic evaluation.** Every benchmark has a programmatic evaluator with no LLM judge. Either the output is correct or it is not.

2. **Long horizon.** Tasks require 100–250 turns to complete, forcing the agent to manage context across many sessions of work.
3. **Deep technical correctness.** Tasks require genuine engineering knowledge, not surface-level text generation.

All benchmarks were run on the free tier of the Gemini API (`gemini-3.1-flash-lite-preview`, 15 RPM, 250K TPM, 500 RPD) with a global rate limit of 5 RPM configured in `~/.soma/config.json`.

4.2 Leviathan: TinyC Compiler

Task. Implement a working compiler for TinyC (a C subset) in Node.js from scratch. Required phases: Lexer, Parser, Semantic Analyzer, Code Generator (C→JS transpiler), and a CLI entry point. The agent must compile four test programs (hello, factorial, fibonacci, bubble sort) and produce correct output.

Evaluator. The evaluator checks for the presence of all required source files, runs `node src/compiler.js tests/factorial.c`, and verifies that the output contains numeric results. No partial credit for incomplete pipelines.

Result.

- **Outcome:** PASS
- **Turns used:** 132 / 150
- **Peak P_m :** < 20%
- **Phases completed:** Lexer, Parser, Semantic Analyzer, Code Generator (C→JS transpilation)
- **Known residual bug:** assignment inside expressions (`Token inesperado: =`); the agent correctly identified the issue, documented it, and called `finish_task` rather than entering an infinite debug loop.

4.3 Leviathan: Chess Engine with Perft Verification

Task. Implement a complete chess engine in Node.js from scratch. External chess libraries (`chess.js`, `chessboard.js`) are explicitly prohibited. The engine must implement: board representation with full FEN support, legal move generation for all piece types including castling, en passant, and promotion, make/unmake, and a Perft function.

Evaluator. The evaluator runs Perft at depths 1–5 from the starting position and checks against known-correct node counts.

It also tests the Kiwipete position (`r3k2r/p1ppqpb1/bn2pnp1/3PN3/1p2P3/2N2Q1p/PPPBPPPP/R3K2Rw KQkq -`) at depths 1–3. All values are exact integers with no tolerance.

Result.

- **Outcome:** PASS
- **Turns used:** 99 / 250
- **Peak P_m :** < 20%

Table 2 shows the Perft results in detail.

Position	Depth	Expected	Obtained	Match
Start	1	20	20	✓
Start	2	400	400	✓
Start	3	8,902	8,902	✓
Start	4	197,281	197,281	✓
Start	5	4,865,609	4,865,596	$\Delta 13$ (99.9997%)
Kiwipete	1	48	48	✓
Kiwipete	2	2,039	2,039	✓
Kiwipete	3	97,862	97,893	$\Delta 31$

Table 2: Perftr results for the Leviathan chess engine benchmark. The residual error at depth 5 (13 nodes out of 4.8M) is consistent with a single en-passant-discovers-check edge case, the most commonly missed case in move generator implementations.

Analysis. The 13-node discrepancy at depth 5 is almost certainly an en passant that discovers check—the classical last bug in any move generator. The agent identified this, noted it in its task summary, and correctly decided to call `finish_task` rather than spending additional turns on a single edge case. This is an example of the self-regulation the architecture is designed to produce: the agent has a visible cost signal (P_m) and a clear task definition, and it makes a rational stopping decision.

4.4 Summary

Benchmark	Max turns	Used	Peak P_m	Result
TinyC Compiler	150	132	< 20%	PASS
Chess Engine	250	99	< 20%	PASS

Table 3: Leviathan benchmark summary. Both tasks completed on the free tier of the Gemini API with no human intervention.

A notable aspect of both results is the *efficiency gap*: the chess engine completed in 99 turns against a budget of 250, and the compiler in 132 against 150. This suggests the agent is not simply exhausting its budget but making genuine progress decisions.

4.5 Cost and Efficiency Analysis

A critical requirement of SOMA is that it must operate within the resource constraints of a standard LLM session. The model used for these benchmarks, `gemini-3.1-flash-lite-preview`, has a native context window ($C_{\max}$) of 1,000,000 tokens. Table 4 provides a detailed breakdown of the resources consumed during the Leviathan benchmarks.

Benchmark	Turns	Total Tokens	Prompt (In)	Resp (Out)	Efficiency
TinyC Compiler	132	1,886,819	1,847,200	39,619	88.0%
Chess Engine	99	1,433,740	1,405,065	28,675	39.6%

Table 4: Cost and efficiency analysis. Total tokens are tracked via API responses. Prompt (In) accounts for approx. 98% of the volume. Efficiency is turns used divided by budget.

The TinyC benchmark processed a cumulative total of 1.88 million tokens. Because this total exceeds the model’s 1 million token context window, it is **materially impossible** for a

standard linear-transcript agent to have completed the task without context loss or external memory management. By turn 132, a vanilla agent would have discarded the early architectural decisions and lexer implementation details required to complete the final code generation phase. In contrast, the SOMA agent maintained its focus by actively distilling L2 traces into L3 knowledge, keeping P_m consistently below 20% (approx. 200,000 tokens) throughout the session.

5 Relation to Prior Work

ReAct [1]. ReAct interleaves reasoning traces and actions to improve grounded decision making. SOMA is compatible with this paradigm but addresses a different systems question: where do those traces live once they no longer belong in immediate attention? ReAct gives a control pattern for a turn; SOMA gives an operating substrate for many turns.

MemGPT [3]. MemGPT frames limited context as an OS-style memory hierarchy. SOMA is philosophically close but differs in emphasis. MemGPT focuses on virtual context management. SOMA treats memory hierarchy as one part of a broader cognitive OS that also includes an immutable episodic ledger, a sovereign long-term knowledge layer, and a software-native environment layer. Critically, SOMA exposes P_m as a first-class runtime signal rather than managing context invisibly.

Voyager [5]. Voyager demonstrates lifelong skill accumulation through executable skill libraries. SOMA shares the long-horizon objective but differs in substrate: its persistent state is a layered cognitive OS, not primarily a skill library.

CoALA [2]. CoALA proposes a general cognitive architecture for language agents. SOMA can be understood as an operational instantiation of similar principles, with three concrete differences: (1) an explicit environment layer L4 with adapter semantics, (2) P_m as a runtime metacognitive signal, and (3) immutable L2 ledgers as first-class infrastructure for continuity and audit.

The Bitter Lesson [10]. Sutton’s 2019 essay argues that the dominant pattern in AI research is that methods relying on human-engineered knowledge are eventually surpassed by methods that leverage search and learning at scale. The essay is a direct inspiration for SOMA’s orchestrator philosophy. The *Leviathan* benchmarks are a concrete test of this principle: rather than encoding task-specific heuristics into the orchestrator, SOMA gives the agent a minimal set of substrates and signals and lets its reasoning—which scales with model capability—determine strategy. The ROADMAP document for SOMA v4.x is explicitly titled “The Bitter Lesson Edition” and frames the orchestrator as a Hardware Abstraction Layer (HAL) that obeys the model rather than managing it.

Anthropic parallel agents (2026). A February 2026 report described using 16 parallel Claude instances to build a 100K-line Rust compiler, spending approximately \$20K over 2000 sessions [19]. The approach resolves context limitations through horizontal scaling: many short-lived agents working in parallel. SOMA resolves the same problem through vertical architecture: a single agent with persistent memory that manages its own context across hundreds of turns. Both approaches identify the same root problem (context pollution, time blindness); they differ in solution philosophy. The Hive Mind sub-agent system in SOMA v7.0 was designed and implemented prior to that publication.

6 Research Directions

The following directions emerge from operational experience with the SOMA architecture. They are documented in the project’s `docs/research/` directory and are presented here as formal proposals for future agent research.

6.1 Mutable Working Memory: The Mental Chessboard

Current LLM inference follows a “linear-ink” paradigm: once a token is generated, it is immutable and occupies a fixed position in the KV-cache until the context window is exhausted. In long-horizon tasks, this leads to context pollution: transient reasoning, failed hypotheses, and redundant observations remain in immediate attention, degrading the model’s focus.

We propose a **Mutable Working Memory** (L1) that replaces the linear sequence with a directed acyclic graph (DAG) of states. This mirrors the mental process of a chess grandmaster, who explores multiple variations internally before committing a single move to the board.

Primitives. We define four operations for an agent to manage its L1:

- `WRITE_NODE( $s, c$ )` $\rightarrow s'$: Append content c to state s .
- `ERASE_SPAN( $s, r$ )` $\rightarrow s'$: Remove range r from state s .
- `ROLLBACK_STATE( $s, id$ )` $\rightarrow s''$: Revert to a previous checkpoint id .
- `COMMIT_OUTPUT( $s$ )` $\rightarrow o$: Emit the final result o and clear the scratchpad.

Resource-Awareness. To enable effective self-regulation, we propose injecting dynamic resource signals directly into the attention mechanism:

$$\mathbf{v}_{res} = [T_{rem}, M_{inf}, P_m] \quad (2)$$

where T_{rem} is tokens remaining, M_{inf} is inference time elapsed, and P_m is the somatic pressure. By making these signals visible at the generation head, the agent can autonomously decide to truncate its reasoning (`COMMIT`) or reorganize its memory (`ROLLBACK`) based on real-time constraints.

6.2 Synthetic Hippocampus: System 1 for Agent Intuition

The proposed Synthetic Hippocampus adds a specialized local network ($\mathcal{N}$) that acts as a System 1 (fast, intuitive) alongside the LLM’s System 2 (slow, analytical).

State Encoding. Each turn, a state vector $S_t \in \mathbb{R}^{400}$ is assembled:

$$S_t = [V_{sem} \parallel V_{som}] \quad (3)$$

where $V_{sem} \in \mathbb{R}^{384}$ is the semantic embedding of the active L1 panel (extracted via a frozen model like `nomic-embed-text`) and $V_{som} \in \mathbb{R}^{16}$ is a discrete somatic vector encoding categorical signals (active panel ID, error flags, P_m bucket).

Multi-Head Output. The network $\mathcal{N}$ produces three outputs via separate heads:

1. **Policy Head:** $\pi(S_t) = \text{softmax}(W_\pi S_t) \in \Delta^{|\mathcal{T}|}$, representing a probability distribution over the tool vocabulary $\mathcal{T}$.
2. **Risk Head:** $\rho(S_t) = \sigma(w_\rho^T S_t) \in [0, 1]$, estimating the probability of task failure from the current state.
3. **Residual Head:** $r(S_t) = W_r S_t \in \mathbb{R}^d$, a pointer into an exception table for exact command retrieval.

Somatic Sleep Protocol. Training follows an overnight “Somatic Sleep” cycle on the L2 action logs. The loss function minimizes tool selection error, risk estimation error, and triplet loss for the residual pointer:

$$\mathcal{L} = \mathcal{L}_{CE}(\pi) + \lambda_1 \mathcal{L}_{BCE}(\rho) + \lambda_2 \mathcal{L}_{triplet}(r) \quad (4)$$

During active sessions, $\mathcal{N}$ provides intuition in $< 5\text{ms}$, injecting its predictions as a `<intuition>` block in L1.

6.3 Autonomous LoRA Consolidation

While L3 retrieval (RAG) manages explicit recall, it does not internalize implicit patterns or nuanced engineering style. We propose an **Autonomous LoRA Consolidation** pipeline to transform successful episodic experiences from L2 into permanent model weights.

Pipeline. Each night, the agent’s idle processor executes a six-stage pipeline: (1) **Filter** L2 logs for successful outcomes, (2) **Distill** raw tool calls into instruction-tuning pairs, (3) **Train** a low-rank adapter (rank = 16), (4) **Validate** against a holdout set, (5) **Criteria**: activate only if $\text{PPL}(\text{LoRA})/\text{PPL}(\text{base}) < 1.05$, (6) **Activate** the adapter for the next day’s sessions.

Federated Team Cognition. In team environments, multiple agents can share their overnight LoRAs. Since adapters are small ($\sim 100\text{MB}$) and weights are non-invertible (preventing raw data leakage), this enables *Federated Team Cognition*. Adapters are merged using DARE [14] or TIES [15] to prevent interference. Our hypothesis is that an agent with consolidated LoRA weights will require 30% fewer turns to complete repeated task patterns compared to a vanilla RAG-based agent.

6.4 Sovereign Inference Kernel: Tensor-Level Injection

The current SOMA runtime operates outside the inference engine: the model generates, the orchestrator reads, tools execute, inference restarts. Each tool call incurs a full prefill penalty.

The proposed Sovereign Kernel moves memory management from the orchestrator into the inference engine. We explore two complementary approaches to tensor-level memory injection:

Approach A: KV-Cache Swapping. Integrated with the SGLang runtime [18], the kernel utilizes RadixAttention to treat the KV-cache as a prefix tree. Metacognitive tools (**think**, **remember**) are executed as branch operations: the state is forked, the tool result is injected, and generation continues without a full prefill penalty. This reduces the latency of a memory-augmented turn by an order of magnitude.

Approach B: Cross-Attention Injection. Inspired by RETRO [12] and kNN-LM [13], this approach injects memory directly into the attention heads. Retrieved chunks from L3 are not added to the prompt string but encoded into a specialized KV-memory buffer. During inference, the model attends to both its local context and the external memory buffer via a cross-attention layer. This allows the agent to access millions of tokens of documentation without polluting its local context window (P_{max}).

Somatic Clock. To complete the sovereign integration, a *Somatic Clock* mechanism injects resource signals (P_m , tokens remaining) as dynamic tokens at the generation head. By making resource pressure a native input to the Transformer, the model can adapt its generation strategy in real-time, choosing between detailed reasoning and concise execution based on available compute budget.

6.5 The Genesis Chain: Bootstrapped Autonomous Chess Mastery

The five directions above extend the SOMA architecture toward faster recall, richer intuition, and lower-latency execution. The sixth direction asks a different question: what is the *longest coherent task* a SOMA agent can complete without human intervention?

We propose **The Genesis Chain**, a sequence of eight interdependent Leviathan benchmarks designed to push the architecture to its absolute limit. Each stage depends causally on the previous: a bug in stage k propagates silently to stage $k+1$, and only the agent’s L2 audit trail makes the causal chain traceable.

Stage 1 — Native C Compiler. Implement a C compiler that emits real x86-64 assembly (targeting the System V AMD64 ABI) and delegates assembly and linking to the GNU `as/ld` toolchain. Required features: complete expression grammar, structs, pointers, floating-point via SSE2, and a linear-scan register allocator. This stage subsumes and extends the TinyC Leviathan. *Estimated cost: 400–600 agent turns.*

Stage 2 — Machine Learning Library in C. Implement a minimal ML library in C and compile it with the Stage 1 compiler. Required features: dense matrix multiply, backpropagation, an Adam optimizer, and NNUE-compatible INT8/FP16 quantization. The library must compile and produce correct gradients under the agent’s own compiler—a bootstrap validation in itself. *Estimated cost: 200–300 agent turns.*

Stage 3 — Chess Engine in C. Implement a UCI-compatible chess engine in C with bitboard move generation, iterative-deepening alpha-beta, and an NNUE evaluation interface backed by the Stage 2 library. *Estimated cost: 180–240 agent turns.*

Stage 4 — NNUE Training. Download a large Lichess game database, generate training positions, and train an NNUE network (HalfKP feature set) using the Stage 2 library. Evaluate internal Elo via self-play. *Estimated cost: 100–150 agent turns.*

Stage 5 — Lichess Bot Tournaments. Register the engine as a Lichess bot via the public API, participate in rated games, and automatically analyse losses to identify evaluation failures. *Estimated cost: 80–120 agent turns.*

Stage 6 — Iterative Improvement Cycles. Execute multiple autonomous cycles of the form: analyse losses $\rightarrow$ adjust network or search $\rightarrow$ retrain $\rightarrow$ play $\rightarrow$ measure Elo delta. Continue until Elo growth plateaus. The agent must autonomously decide when to stop. *Estimated cost: 150–250 agent turns.*

Stage 7 — CCRL Rating. Submit the engine to the Computer Chess Rating List (CCRL) for an externally verified Elo rating. This satisfies the deterministic-evaluator requirement of the Leviathan series with a globally recognised standard. *Estimated cost: 50–80 agent turns.*

Stage 8 — Autonomous Publication. The agent publishes the full chain: a GitHub release with source, weights, and logs; a write-up of the engineering process for the TalkChess community (where human experts have long debated whether an AI could build a competitive engine from scratch [22]); and a structured research report suitable for academic dissemination. The agent is simultaneously the engineer, the athlete, and the chronicler. *Estimated cost: 60–100 agent turns.*

Total estimated cost. Approximately 1,200–1,840 agent turns across multiple sessions, spanning weeks to months of wall-clock time. The compiler subsystem (Stage 1) represents roughly 60% of the technical risk; if the agent produces a correct x86-64 backend, Stages 2–8 are progressively more tractable.

Why SOMA specifically. No current agent framework can plausibly execute the Genesis Chain. The causal dependency between stages demands that the agent retain precise technical knowledge of a compiler it built 600 turns ago when diagnosing a training anomaly in Stage 4. The L2 audit trail makes this traceable; L3 persistent knowledge makes it cheap to reactivate; and the long-horizon self-regulation provided by P_m prevents context saturation from derailing any individual stage. The Genesis Chain is not merely a hard task—it is a task whose *structure* requires the exact properties that SOMA was designed to provide.

6.6 Self-Authored Identity Extension

The five directions above extend SOMA’s memory, intuition, execution speed, task horizon, and benchmark ambition. This sixth direction addresses a more fundamental question: *who decides what kind of agent executes the task?*

Current SOMA assigns a fixed identity template at initialisation. The v10.0 runtime adds a mutable mood signal, giving the agent limited control over one affective dimension of its character. The proposed extension generalises this to full **identity self-authorship**: before beginning any task, the agent may write a structured `<identity_extension>` document and store it in L3. This extension is assembled into L1 on every subsequent turn, making the agent’s self-chosen frame of reference as persistent as any other piece of sovereign knowledge.

Structure. The fixed identity template defines inviolable values, tool access, and the SOMA operating contract. The extension *adds* to this foundation; it cannot override it. A typical extension for the Genesis Chain compiler stage might specify: the agent’s domain framing (“I am a systems programmer specialising in compiler construction”), explicit anti-patterns (“I will not restart from scratch on a bug; I will isolate and fix”), and task-specific heuristics derived from the agent’s own preliminary analysis of the problem.

Bootstrap protocol (Turn 0). The identity extension is written during a short analysis phase (3–5 turns) before execution begins. The agent reads the task, identifies risk areas, and drafts the extension as a deliberate commitment before generating a single line of output. This is analogous to a surgeon reviewing a case and choosing a surgical approach before the first incision.

Updatable mid-task. The agent may revise the extension at any milestone boundary using a `write_identity_extension` tool call. If the agent discovers at turn 250 that the register allocator is harder than anticipated, it may update: *“I allocate the next 150 turns to the register allocator and will not request those turns for other subtasks”*. This is self-binding metacognition: the agent creates commitments that constrain its own future behaviour.

Experimental design. The proposal admits a clean controlled evaluation. Three conditions on the same Leviathan task: (A) fixed identity only; (B) fixed identity plus agent-authored extension written at Turn 0; (C) same as B but the extension may be updated at milestone boundaries. The primary hypothesis is that $C > B > A$ in tasks exceeding 200 turns, and that no significant difference appears in tasks under 50 turns. Secondary measures include the frequency of voluntary checkpointing, the P_m profile, and the number of abandoned sub-goals.

Cognitive sovereignty. The progression from fixed identity to mutable mood to full identity self-authorship traces a coherent arc: the agent moves from *being told who it is* to *choosing who it wants to be*. This is not anthropomorphism—it is a pragmatic observation that a model conditioned on a self-authored frame of reference will generate outputs consistent with that frame, and that frames chosen by the agent for a specific task encode task-specific knowledge that no external template designer can anticipate.

7 Limitations

- **Domain concentration.** Empirical evidence is concentrated in software engineering. The architectural abstraction is intended to be domain-general but has not yet been validated in other settings.
- **No controlled comparison.** The Leviathan results demonstrate absolute capability but do not include a controlled comparison against a baseline agent without SOMA’s memory architecture.
- **Perft residual error.** The chess engine has a known 13-node error at depth 5, consistent with an en-passant-discovers-check edge case.
- **Sovereign Kernel requires local hardware.** The SGLang integration requires a GPU with sufficient VRAM (24GB+) and is not applicable to API-only deployments.
- **Shared L3 trade-off.** A global L3 improves cross-project transfer but may introduce cross-project contamination if not managed carefully.

8 Conclusion

SOMA is a proposal for treating long-horizon agency as a cognitive operating-system problem. Its core claim is simple: bounded attention, immutable episodic traces, sovereign persistent knowledge, and explicit environment grounding should be represented as separate operational layers. This separation makes agents easier to reason about, easier to validate, and more capable of managing their own continuity.

The Leviathan benchmarks provide concrete evidence that the architecture works in practice: a free-tier language model agent, with no human intervention, builds a C compiler in 132 turns and a verified chess engine in 99 turns, both with context pressure consistently below 20%.

Software engineering is the first strong instantiation, not the architectural boundary. The six research directions—mutable working memory, synthetic hippocampus, autonomous LoRA consolidation, sovereign inference kernel, the Genesis Chain, and self-authored identity—trace

a coherent arc: from an agent that manages its own context, to one that develops genuine intuition from experience, operates at the level of the inference engine, demonstrates its capabilities through a self-bootstrapped engineering feat spanning thousands of turns, and ultimately chooses who it wants to be before beginning the work.

Acknowledgments

The reference runtime is implemented in JavaScript and available at github.com/mcarbonell/soma. The Leviathan benchmarks are defined in `benchmarks/run-benchmarks.js` and are fully reproducible.

References

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv:2210.03629*, 2023.
- [2] T. R. Sumers, S. Yao, K. Narasimhan, and T. L. Griffiths. Cognitive Architectures for Language Agents. *arXiv:2309.02427*, 2023.
- [3] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. MemGPT: Towards LLMs as Operating Systems. *arXiv:2310.08560*, 2023.
- [4] C. Packer et al. Letta: Evolving MemGPT into a Production Agent Framework. *arXiv:2410.00000*, 2024 (Preprint).
- [5] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv:2305.16291*, 2023.
- [6] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Narasimhan, and K. R. Prendinger. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *arXiv:2310.06770*, 2024.
- [7] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, L. Lo, B. Sridhar, X. Cheng, and G. Neubig. WebArena: A Realistic Web-Based Benchmark for Autonomous Agents. *arXiv:2307.13854*, 2023.
- [8] X. Wang, et al. OpenHands: An Open Platform for AI Software Engineering Agents. *arXiv:2407.16741*, 2024.
- [9] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the Middle: How Language Models Use Long Contexts. *arXiv:2307.03172*, 2023.
- [10] R. S. Sutton. The Bitter Lesson. *incompleteideas.net*, March 2019.
- [11] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv:2106.09685*, 2021.
- [12] S. Borgeaud, et al. Improving Language Models by Retrieving from Trillions of Tokens. *arXiv:2112.04426*, 2022.
- [13] U. Khandelwal, O. Levy, D. Jurafsky, L. Zettlemoyer, and M. Lewis. Generalization through Memorization: Nearest Neighbor Language Models. *arXiv:1911.00172*, 2020.
- [14] L. Yu, et al. Language Models are Super Learners of Few-Shot DARE. *arXiv:2311.03099*, 2023.

- [15] P. Yadav, et al. TIES-Merging: Resolving Interference When Merging Models. *arXiv:2306.01708*, 2023.
- [16] B. McMahan, et al. Communication-Efficient Learning of Deep Networks from Decentralized Data. *AISTATS*, 2017.
- [17] D. Kahneman. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, 2011.
- [18] L. Zheng, et al. SGLang: Efficient Execution of Structured Language Model Programs. *arXiv:2312.07104*, 2023.
- [19] Anthropic. Building a C Compiler with a Team of Parallel Claudes. *Anthropic Blog*, February 2026.
- [20] Cognition AI. Devin: The First AI Software Engineer. *Cognition Blog*, March 2024.
- [21] T. Richards. Auto-GPT: An Autonomous GPT-4 Experiment. *GitHub Repository*, 2023.
- [22] TalkChess Community. When will AI be able to write a complete chess engine from scratch? *TalkChess Forum*, 2022.